Right from the Start: enabling Responsible Programmers to begin with correct specifications

Secure Program Synthesis Fellowship (SPS) sponsored research on the "Specification Elicitation" problem

October 2026

Denis Papathanasiou
denis@papathanasiou.org

Siddhartha Prasad
Brown University
siddhartha_prasad@brown.edu

Shriram Krishnamurthi
Brown University
shriram@brown.edu

Abstract

Even as artificial intelligence (AI) aggressively reshapes the modern software development lifecycle (SDLC), capturing requirements remains a fundamental activity. Starting with a formal specification that can be verified is particularly advantageous in an environment where source code generation is cheap, but supporting and maintaining that code can be expensive.

Software requirements in industry, however, are still communicated mostly in plain (human) language, which can be vague, imprecise, and subject to misinterpretation when it comes to implementation. We propose a new method of leveraging AI to solicit requirements, inspired by the Pairwise Iterative-Choice Knockout (PICK) workflow, which relies on human feedback at critical points in the process.

Our prototype enables people to write requirements in plain language, while maintaining a corresponding formal model in parallel, without having to be expert in the underlying formal syntax.

---

1. Background

1.1 The current AI-Driven SDLC

Two influential blog posts appeared within a few weeks of each other at the end of 2025, Martin Kleppmann's *Prediction: AI will make formal verification go mainstream*, and Steve Yegge's *Welcome to Gas Town*, an ode to agentic "*vibe coding*", in all its messy glory.

Despite the snark and antagonism of the latter, it is currently winning the day with large software companies, as they race to spend significant amounts of money on large language model (LLM) tokens, without any expectation of actual return on investment (ROI), a shocking exception to how captial expenditure is usually allocated.

In the current environment, there is relatively little attention being paid to the ideas in Kleppmann's post, and even code reviews and unit testing are getting short shrift in the rush to build our own Gas Towns.

This project seeks to provide an antidote, in the form of intuitive tools and processes for the Responsible Programmers (RP) in corporate environments, such that they can start with the right requirements before committing time, effort, or tokens, on building the wrong thing.

1.2 The Requirements Gathering Process

To the extent that software professionals working in large corporate environments have heard of formal methods at all, they tend to avoid using them in practice, given the perceived poor cost-benefit tradeoff.

Requirements are therefore still specified in human language, sometimes written by people who lack the training or ability to do the implementation, which can lead to significant divergence in terms of what is intended versus what gets produced.

This is not a new phenomenon: the proverbial tire swing cartoon illustrating this gap actually originated in 1973!

---

2. Introduction

2.1 High-Level Approach

"*A Whirlwind Tour*", the second chapter of *Software Abstractions*, Daniel Jackson's introductory text on Alloy, describes how a formal model can be constructed incrementally, with immediate visual feedback given at each stage, allowing the modeler to both confirm and reconsider basic ideas and rules, as the model evolves.

Alloy is uniquely suitable for this type of exploration because it does not require all the model properties to be defined in full, before its analyzer can produce meaningful results.

2.2 Objectives

The aim of this project is to reify that process, to provide the RP with better means for conducting requirements interviews with business sponsors, since that is already part of the current SDLC, and it is also unambiguously perceived as adding value.

More specifically, this project aims to build a working prototype of an interactive requirements gathering tool, which is intuitive and accessible, even to end-users without a background in formal methods.

The goal is to provide an app capable of accepting software requirements in human language, and provide immediate user feedback in terms of correctness and consistency, all while accumulating the corresponding model definitions in Alloy syntax "under the covers", as a by-product of those interactive user sessions.

---

3. Technical Architecture

3.1 Application Components

1. Algorithm Bot (or "*algobot*" for short) is a large language model (LLM), supplemented with a retrieval augmented generation (RAG) pipeline; and
2. Tools are deterministic functions:
    - Model Evaluator invokes Alloy as a library
    - Model Visualizer uses Spytial to produce interactive diagrams

- Semantic Database is a queryable store of all the model changes, enabling not only undo/redo actions but also compositions across versions

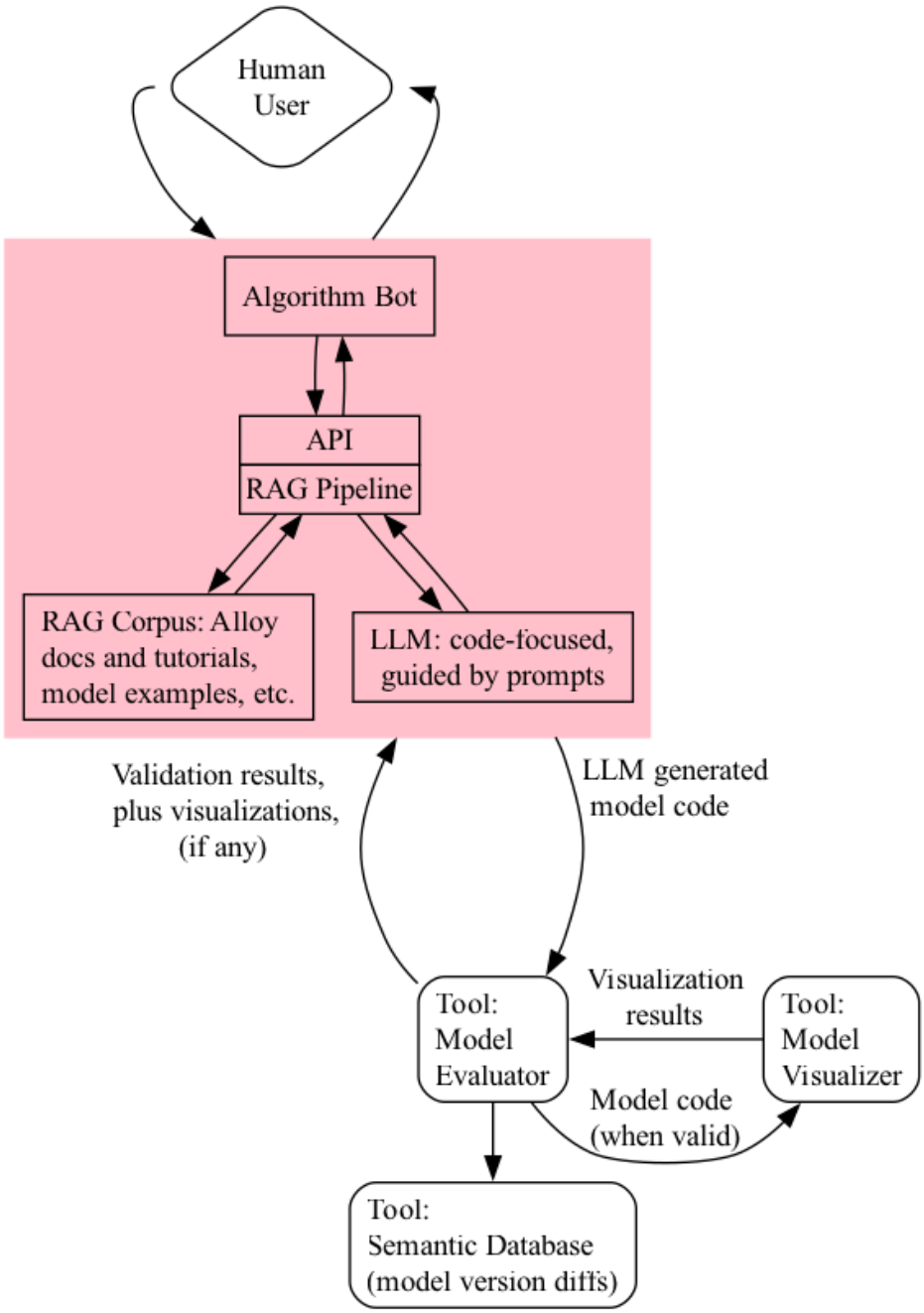


Figure 1: Architecture Diagram

3.2 Workflow

Inspired by PICK, this a validation workflow, with a human in the loop making decisions to accept or reject the current state:

- A user provides inputs, in the form of plain language requirements
- Algobot evaluates those inputs, and attempts to generate the corresponding formal model
- Only when the formal model passes a deterministic evaluation (i.e., the LLM has produced a syntactically valid model) does the user get to see the result:
  - The result may be valid syntactically, but contain a verification flaw, indicating that the requirements are in conflict, and need further refinement from the user; or
  - Even if the result is correct both in terms of syntax and verification, the model may not be what the user intended, and so the user gets to vote "*thumbs down*," and add more details; or
  - The result is both correct and as intended, so voting "*thumbs up*" produces the formal model and plain language requirements text, side by side
- At each step, the user can decide to continue making revisions, or stop
- All of this history is persisted in the version database, which can also be queried and analyzed outside of this interactive session

3.3 Demonstration

This video demonstrates the prototype in action, taking a basic example from Jackson, to define a simple concept.

We used the Kimi K2.7 Code LLM for this demonstration, supplemented with Alloy model examples and the Alloy grammar definitions, both the official sources, and the experimental ANTLR file (Alloy.g4) as the RAG corpus.

The source code for this project is here:

https://github.com/dpapathanasiou/sps-specification-elicitation

---

4. Conclusion

This research proposes a solution for enabling the use of formal methods for writing software specifications in enterprise environments: by combining the strengths of generative AI with human decision making and judgement, we make writing proper specifications more accessible, to a wider audience of stakeholders.